

PRACTICAL TAKE-HOME ASSIGNMENT

Demoblaze QA Automation

Software Quality Assurance Analysis, Manual Test Design
and Selenium WebDriver Implementation

Report detail	Value
Student	Bavindu Shamen
System Under Test	Demoblaze E-Commerce Store
URL	https://www.demoblaze.com/
Technology	Java 21 Selenium 4.44 TestNG 7.12 Maven POM
Verified execution	13 August 2026 5 passed 0 failed 0 skipped

CURRENT RESULT: BUILD SUCCESS

Prepared from the current Markdown answers, implementation, Git history, and a fresh local Selenium execution.

Executive summary

This report documents the analysis and automation of the public Demoblaze e-commerce application. The project covers website selection, requirements and risks, fifteen manual test scenarios, five risk-based automation decisions, Page Object Model architecture, Selenium/TestNG implementation, execution evidence, debugging, and AI usage disclosure.

The current suite was rerun on 13 August 2026. All five selected tests passed in headless Chrome: five tests, zero failures, zero errors, and zero skipped. The result validates invalid-login handling, AJAX category filtering, adding and removing a cart item, and completing a synthetic order.

The application replaced the initial nopCommerce choice after Cloudflare security verification prevented reliable browser automation. Demoblaze did not present that blocker during the current execution and supplied the dynamic catalog, modal, alert, cart, and checkout behaviours needed for the assignment.

1. Website selection

Item	Decision
Application	Demoblaze E-Commerce Store
URL	https://www.demoblaze.com/
Type	Business-to-consumer e-commerce demo
Environment	Public demonstration environment
Stack	Java, Selenium WebDriver, TestNG, Maven, Page Object Model

Selection rationale

Demoblaze provides a realistic customer journey without requiring production access or real payment information. Its feature set includes user authentication, category filtering for phones, laptops and monitors, product details, cart management, deletion, checkout form entry, JavaScript alerts, and order-confirmation dialogs.

The site naturally separates into reusable page and modal objects and exposes both positive and negative test paths. During the verified run it supported stable headless execution, making it a better automation target than the original site.

Scope

In scope	Out of scope
Home and category navigation	Real financial transactions
Negative login validation	External email delivery
Product details and pricing	Performance and load testing
Cart add/remove behaviour	Administration backend
Synthetic checkout and confirmation	Production data or accounts

2. Requirement analysis

Application objectives

- Provide an intuitive electronics catalog organised by category.

- Allow visitors to view product information and add or remove shopping-cart items.
- Support a synthetic checkout and display confirmation metadata.
- Provide sign-up and login interactions with clear modal or alert feedback.

Key functional areas

ID	Area	Expected behaviour
FR-01	Catalog	Display products and filter the grid by Phones, Laptops, or Monitors.
FR-02	Product	Display name, price, description, and an Add to cart action.
FR-03	Cart	Persist item titles and prices, calculate totals, and allow deletion.
FR-04	Checkout	Collect buyer data and display purchase confirmation with order metadata.
FR-05	Authentication	Accept modal input and provide JavaScript alert feedback.

Quality risks and mitigations

Risk	Impact	Mitigation
AJAX delay during category filtering	High	Wait for stale product elements and repoll the refreshed grid.
Delayed cart DOM population	High	Wait for cart.html and query rows only after the dynamic response.
JavaScript alerts block WebDriver	Medium	Wait for alertIsPresent, capture its text, then accept it.
Shared public demo state	Medium	Create prerequisites within each test and use synthetic input.
Browser/CDP version drift	Low	Use Selenium Manager; record warnings separately from functional outcomes.

Assumptions

- The public site remains reachable over HTTPS.
- Chrome executes JavaScript and supports both headless and headed modes.
- Catalog and cart content are rendered asynchronously using client-side requests.
- Synthetic checkout does not cause a real charge or fulfilment request.

3. Manual test scenario design

The following fifteen scenarios cover authentication, catalog discovery, product details, cart state, checkout validation, navigation, contact, and media interaction. High priority denotes a core security, cart, or checkout journey; Medium covers important supporting behaviour; Low covers informational or convenience behaviour.

ID	Module	Scenario	Priority	Execution
MTS-01	Auth	Reject invalid login credentials	High	AUT-01
MTS-02	Catalog	Filter products by Laptops category	Medium	AUT-02
MTS-03	Cart	Add a product to the shopping cart	High	AUT-03
MTS-04	Cart	Remove a product from the shopping cart	High	AUT-04
MTS-05	Checkout	Complete guest order with valid synthetic details	High	AUT-05
MTS-06	Auth	Sign up with an existing username	Medium	Manual
MTS-07	Catalog	Filter products by Phones category	Medium	Manual
MTS-08	Catalog	Filter products by Monitors category	Medium	Manual
MTS-09	Product	View details for Sony vaio i5	Low	Manual
MTS-10	Cart	Calculate total price for multiple items	Medium	Manual
MTS-11	Cart	Open an empty cart	Low	Manual
MTS-12	Checkout	Submit checkout with blank required fields	High	Manual
MTS-13	Navigation	Return Home using the brand link	Low	Manual
MTS-14	Contact	Open the Contact modal	Low	Manual
MTS-15	About Us	Play video in the About Us modal	Low	Manual

Detailed scenarios

MTS-01 - Reject invalid login credentials

Priority: High Preconditions: Visitor is on the Home page.

Steps: Open Log in; enter invalid_user and wrong_pass; submit.

Expected result: An alert states that the user does not exist or the password is wrong, and no authenticated state is created.

MTS-02 - Filter products by Laptops category

Priority: Medium Preconditions: Home catalog is loaded.

Steps: Select Laptops from the category sidebar; wait for the grid to refresh.

Expected result: Only laptop products are displayed and Sony vaio i5 is present.

MTS-03 - Add a product to the shopping cart

Priority: High Preconditions: Home page is available and the cart prerequisite is controlled.

Steps: Open Samsung galaxy s6; select Add to cart; accept Product added alert; open Cart.

Expected result: The cart contains Samsung galaxy s6 with its displayed price.

MTS-04 - Remove a product from the shopping cart

Priority: High Preconditions: Samsung galaxy s6 has been added during this test flow.

Steps: Open Cart; select Delete for Samsung galaxy s6; wait for the row to disappear.

Expected result: The product is absent and the cart state/total is updated.

MTS-05 - Complete guest order with valid synthetic details

Priority: High Preconditions: A product exists in the cart.

Steps: Open Cart and Place Order; enter synthetic name, country, city, card, month, and year; select Purchase.

Expected result: A Thank you for your purchase confirmation displays an order ID and amount.

MTS-06 - Sign up with an existing username

Priority: Medium Preconditions: A known demo username has already been registered.

Steps: Open Sign up; enter the existing username and a password; submit.

Expected result: An alert explains that the user already exists and no duplicate account is created.

MTS-07 - Filter products by Phones category

Priority: Medium Preconditions: Home catalog is loaded.

Steps: Select Phones and wait for the product grid to refresh.

Expected result: The refreshed grid contains phone products and excludes laptop/monitor-only results.

MTS-08 - Filter products by Monitors category

Priority: Medium Preconditions: Home catalog is loaded.

Steps: Select Monitors and wait for the grid to refresh.

Expected result: The refreshed grid contains monitor products such as Apple monitor 24.

MTS-09 - View details for Sony vaio i5

Priority: Low Preconditions: Laptops are visible.

Steps: Open Sony vaio i5 from the grid.

Expected result: The details page shows the correct name, price, description, and Add to cart action.

MTS-10 - Calculate total price for multiple items

Priority: Medium Preconditions: Two known products have been added to the cart.

Steps: Open Cart; record each line price; compare their sum with the displayed total.

Expected result: The total equals the sum of the displayed item prices.

MTS-11 - Open an empty cart

Priority: Low Preconditions: No items have been added in the current clean session.

Steps: Select Cart from the navigation bar.

Expected result: The cart page opens without item rows and shows a zero/blank total state.

MTS-12 - Submit checkout with blank required fields

Priority: High Preconditions: A product exists in the cart and Place Order is open.

Steps: Leave required buyer fields blank and select Purchase.

Expected result: The order is not completed and a validation alert requests the required information.

MTS-13 - Return Home using the brand link

Priority: Low Preconditions: Visitor is on a product or cart page.

Steps: Select the PRODUCT STORE brand link.

Expected result: The browser returns to the Home catalog and products become visible.

MTS-14 - Open the Contact modal

Priority: Low Preconditions: Home page is available.

Steps: Select Contact from the navigation bar.

Expected result: A Contact modal appears with email, name, message, Send message, and Close controls.

MTS-15 - Play video in the About Us modal

Priority: Low Preconditions: Home page is available and media playback is permitted.

Steps: Open About us; select the video play control; then close the modal.

Expected result: The modal opens, the video begins playback, and closing returns control to the page.

4. Automation decision

The selected set balances a negative authentication check, asynchronous catalog behaviour, two cart state transitions, and the end-to-end purchasing journey. The remaining scenarios have lower regression return, duplicate category mechanics, or require human visual/media judgement.

Automation	Manual	Scenario	Selection rationale
AUT-01	MTS-01	Invalid login	Negative security boundary; stable alert assertion.
AUT-02	MTS-02	Laptops filter	Core AJAX navigation and refreshed-grid validation.
AUT-03	MTS-03	Add to cart	Primary e-commerce state transition and persistence.
AUT-04	MTS-04	Remove from cart	High-value DOM deletion and cart-state update.
AUT-05	MTS-05	Guest checkout	End-to-end modal form and confirmation journey.

Exclusion rationale

- MTS-06 depends on a persistent username and shared demo state.
- MTS-07 and MTS-08 repeat the category mechanism already covered by AUT-02.
- MTS-09, MTS-13, and MTS-14 are inexpensive to verify manually.
- MTS-10 and MTS-12 are strong future automation candidates after the core suite.
- MTS-15 requires visual/media judgement beyond the initial functional regression scope.

5. Framework design

The framework uses Java 21, Selenium WebDriver 4.44.0, TestNG 7.12.0, Maven 3.9+, and Page Object Model. The Java package retains the legacy `lk.ucsc.nopcommerce` namespace after the SUT migration; behaviour and configuration now target Demoblaze.

Component	Responsibility
ConfigReader	Loads <code>config.properties</code> and system-property overrides.
DriverFactory	Owns <code>ThreadLocal</code> WebDriver lifecycle and Chrome options.
BasePage	Provides explicit waits, interactions, and alert handling.
HomePage	Navigation, category filtering, product discovery, and modal access.
ProductPage	Product detail verification and Add to cart action.
CartPage	Cart rows, item deletion, total state, and checkout trigger.
LoginModal	Invalid credential submission and alert capture.
PlaceOrderModal	Synthetic buyer form and purchase confirmation.
CustomerOrder	Immutable synthetic order data.
BaseTest / listener	Per-test browser isolation, teardown, and failure screenshots.

Key design controls

- Locators and interactions are isolated in Page Objects; TestNG tests own assertions.
- `WebDriverWait` handles AJAX re-rendering, URL changes, visibility, clickability, and alerts.
- Each test starts with a fresh browser to prevent cart or authentication state leakage.
- A failure listener writes timestamped screenshots for root-cause analysis.

6. Automated implementation

ID	Page flow	Principal assertion
AUT-01	HomePage -> LoginModal	Alert contains user-not-found or wrong-password feedback.
AUT-02	HomePage category filter	Refreshed product names contain Sony vaio i5.
AUT-03	HomePage -> ProductPage -> CartPage	Cart contains Samsung galaxy s6.
AUT-04	ProductPage -> CartPage deletion	Cart no longer contains Samsung galaxy s6.
AUT-05	CartPage -> PlaceOrderModal	Confirmation thanks the buyer and includes order metadata.

Test data and independence

Changeable catalog and credential data live in config.properties. Invalid login uses synthetic credentials. Checkout uses CustomerOrder.validSyntheticOrder(), avoiding personal or real payment information. Add, remove, and checkout scenarios create their own cart prerequisites within a new browser session.

7. Test execution

Execution detail	Observed value
Date/time	13 August 2026, 00:57 Asia/Colombo
Mode	Headless Chrome through Selenium WebDriver
Tests	5
Failures / Errors / Skips	0 / 0 / 0
Suite time	44.60 seconds
Maven result	BUILD SUCCESS

VERIFIED: 5 PASSED | 0 FAILED | 0 ERRORS | 0 SKIPPED

Execution note

Selenium emitted a warning that Chrome DevTools Protocol version 151 did not have an exact Selenium match and used the nearest available version 148. The suite does not depend on DevTools-specific APIs, so this warning did not affect the five WebDriver tests or their passing result. It should be rechecked after a Selenium upgrade.

8. Debugging challenge

Initial symptom

The original nopCommerce suite timed out while waiting for application elements because a Cloudflare security-verification page replaced the expected storefront UI in automated Chrome sessions.

Diagnosis and corrective work

- Failure screenshots revealed the security-verification page before the first application interaction.
- The SUT was changed to Demoblaze, a demo environment that allowed the required browser flows during execution.

- Page Objects were redesigned for Demoblaze modals, alerts, dynamic catalog rendering, cart rows, and checkout.
- Explicit waits for staleness, URLs, elements, and alerts resolved AJAX timing and stale-element failures.

Verification

After the migration and synchronization fixes, the fresh headless run completed with five tests and no failures, errors, or skips. The result shows that the environmental blocker and subsequent test-synchronization issues were correctly separated and addressed.

9. Maintainability and limitations

Page Objects localise UI changes, configuration externalises mutable data, and clean WebDriver sessions protect independence. The most important ongoing risk is reliance on a shared public demo whose content and availability are outside the project team's control. Test results should always be interpreted alongside the first observed divergence and the captured evidence.

The package/artifact names still reference nopCommerce because the project migrated after repository initialization. This does not affect execution, but renaming the Java namespace and Maven artifact to demoblaze would improve clarity in a future cleanup.

10. Conclusion

The Demoblaze project satisfies the assignment's core deliverables: a justified website choice, requirement and risk analysis, fifteen manual scenarios, five automation decisions, a Java/Selenium/TestNG Page Object framework, five implemented browser tests, a debugging narrative, meaningful Git history, and an AI disclosure. The latest execution produced BUILD SUCCESS with all five selected scenarios passing.

Appendix A - Key migration commits

Commit	Milestone
63982d0	Migrate the automated suite to Demoblaze and implement the new Page Objects.
ad8fa7a	Refactor cart product-name queries for dynamic DOM handling.
9e06d2b	Add explicit waits and stale-element handling for navigation and listings.

Appendix B - AI usage disclosure

The Markdown disclosure states that Antigravity AI (Google DeepMind) assisted with target evaluation, Cloudflare diagnosis, Page Object Model architecture, the Demoblaze refactor, AJAX/alert synchronization, and documentation. The student retains responsibility for reviewing and understanding the implementation, executing it, explaining the decisions during the viva, and recording the five-minute reflection.

Source basis

This report was prepared from README.md, AI_USAGE.md, docs/01 through docs/05, docs/07, the current Java/configuration files, Git history, and the fresh Maven/Surefire execution on 13 August 2026. The older docs/06 nopCommerce execution record was not used as the current Demoblaze result.